Sudoku Intelligence Lab

Engineering Benchmark Report

An Empirical Performance Analysis of an Instrumented Recursive Backtracking Sudoku Solver

Author:
Navya Nawal

Department of Computer Science & Engineering
BMS Institute of Technology &Management

GitHub Repository:
Sudoku Intelligence Lab - Github Repository

Year
2026

Abstract

Sudoku is commonly used as a benchmark problem for evaluating search algorithms due to its well-defined constraints and varying levels of computational complexity. While many implementations focus primarily on solving puzzles, comparatively little attention is given to understanding the internal behavior of the solving algorithm itself.
This project presents **Sudoku Intelligence Lab**, an engineering framework for benchmarking and analyzing an instrumented recursive backtracking Sudoku solver implemented in C. Rather than treating the solver as a black box, the system records detailed execution metrics including recursive calls, backtracks, candidate checks, successful and failed assignments, recursion depth, execution time, and puzzle density across a benchmark dataset of 1,000 Sudoku puzzles.
The generated performance data is processed using a Python-based analytics pipeline and examined through descriptive statistics, regression analysis, and prediction models. An interactive Streamlit dashboard provides visual exploration of solver behavior, enabling users to investigate relationships between puzzle characteristics and computational effort.
Experimental results demonstrate that recursive call count is a highly reliable predictor of execution time, while the number of empty cells alone is insufficient to estimate solving complexity, highlighting the importance of puzzle structure in recursive search algorithms. By combining algorithm implementation, automated benchmarking, statistical analysis, and interactive visualization, Sudoku Intelligence Lab demonstrates an end-to-end engineering workflow for empirical performance evaluation and serves as a reusable framework for studying algorithmic behavior beyond traditional correctness testing.

Keywords

Recursive Backtracking,
Sudoku,
Constraint Satisfaction,
Algorithm Benchmarking,
Performance Instrumentation,
Statistical Analysis,
Regression,
Streamlit

Introduction

Sudoku is one of the most widely recognized constraint satisfaction problems (CSPs) and has long served as a practical benchmark for evaluating search algorithms. Although the rules of Sudoku are straightforward, the computational effort required to solve different puzzles varies considerably depending on the arrangement of clues and the complexity of the search space.

Recursive backtracking is among the most commonly used techniques for solving Sudoku. The algorithm systematically explores candidate values, backtracking whenever a constraint violation is encountered, until a valid solution is found. While recursive backtracking is conceptually simple and guarantees correctness for valid puzzles, its computational performance depends heavily on the structure of the puzzle being solved. Two puzzles containing a similar number of empty cells may require vastly different numbers of recursive calls and backtracking operations before reaching a solution.

Traditional Sudoku solver implementations primarily focus on correctness and execution speed. Once a puzzle is solved, little attention is given to understanding *how* the solver reached that solution or which internal operations contributed most to the computational cost. As a result, valuable information about algorithmic behavior remains unexplored.

This report presents **Sudoku Intelligence Lab**, an engineering benchmark framework designed to analyze the internal execution characteristics of a recursive backtracking Sudoku solver. Rather than viewing the solver solely as a tool for producing solutions, the project instruments the algorithm to record detailed performance metrics throughout execution. These measurements enable a systematic investigation of solver behavior across a large benchmark dataset, providing insights into the relationship between puzzle characteristics, search complexity, and computational effort.

The project integrates algorithm implementation, automated benchmarking, statistical analysis, and interactive visualization into a single engineering workflow. By combining these components, the proposed system demonstrates how empirical performance analysis can complement traditional algorithm implementation, transforming a classical programming exercise into a reproducible experimental framework.

Motivation

Many algorithm implementations emphasize producing correct outputs while providing limited visibility into the computational process that generated those outputs. Although recursive backtracking has been extensively studied as a general search strategy, practical implementations often treat the solver as a black box, reporting only whether a puzzle was solved and how long execution required. Such measurements reveal little about the underlying search process or the factors responsible for differences in computational effort.

This project was motivated by the desire to move beyond correctness testing and toward **empirical performance analysis**. Instead of asking only whether a Sudoku solver can solve a puzzle, the project investigates how the solver behaves internally while solving puzzles of varying complexity. Questions such as *How many recursive calls were required?*, *How frequently did the algorithm backtrack?*, *Does puzzle density accurately predict computational effort?*, and *Which execution metrics best explain runtime?* formed the foundation of the study.

To answer these questions, the recursive backtracking solver was instrumented to capture detailed execution metrics for every benchmark puzzle. These measurements were then processed using an automated analytics pipeline that produced statistical summaries, regression models, performance predictions, and interactive visualizations. The resulting framework transforms raw algorithm execution into structured experimental data suitable for engineering analysis.

Beyond Sudoku itself, the project demonstrates a broader engineering methodology for studying algorithms. By integrating implementation, benchmarking, data collection, statistical analysis, and visualization within a single workflow, the framework illustrates how software engineering and empirical experimentation can be combined to better understand computational systems.

Objectives

The primary objective of Sudoku Intelligence Lab is to develop an engineering framework for empirically evaluating the performance of a recursive backtracking Sudoku solver. Rather than measuring correctness alone, the project seeks to quantify the computational effort required to solve puzzles of varying complexity through systematic instrumentation and analysis.

The specific objectives of the project are:

- Design and implement a recursive backtracking Sudoku solver in C capable of solving standard 9×9 Sudoku puzzles.
- Instrument the solver to capture detailed execution metrics, including recursive calls, backtracks, candidate checks, successful assignments, failed assignments, recursion depth, execution time, and puzzle density.
- Generate and benchmark a dataset of 1,000 Sudoku puzzles representing multiple difficulty levels.
- Develop an automated Python pipeline to process solver outputs into structured datasets suitable for statistical analysis.
- Perform descriptive statistical analysis to investigate relationships between puzzle characteristics and solver performance.
- Evaluate regression and interpolation techniques for predicting computational effort from recorded performance metrics.
- Design an interactive Streamlit dashboard that enables exploration of benchmark results and experimental findings.
- Demonstrate an end-to-end engineering workflow integrating algorithm implementation, benchmarking, analytics, visualization, and technical reporting.

Collectively, these objectives establish this Sudoku implementation as a benchmarking and analysis framework rather than solely a Sudoku solving application.

System Overview

Sudoku Intelligence Lab is an integrated software system designed to benchmark and analyze the computational behavior of a recursive backtracking Sudoku solver. The project combines algorithm implementation, automated experimentation, statistical analysis, and interactive visualization into a unified engineering workflow.

The system consists of four major components:

Algorithm Engine.
The core solver is implemented in C using a recursive backtracking algorithm. During execution, the solver records multiple performance metrics that characterize the search process rather than simply producing the solved puzzle.

Analytics Pipeline.
Benchmark results generated by the solver are processed using Python. The analytics pipeline converts raw execution data into structured datasets and performs statistical analysis, regression modeling, interpolation-based prediction, and validation experiments.

Interactive Dashboard.
A Streamlit dashboard provides an interactive interface for exploring benchmark results. Users can inspect individual puzzles, visualize solver performance, examine statistical relationships, evaluate prediction models, and review the principal findings derived from the experimental data.

Engineering Documentation.
The project includes a structured repository, automated reports, visualizations, and this engineering benchmark report to ensure that the methodology, implementation, and experimental results are reproducible and accessible.

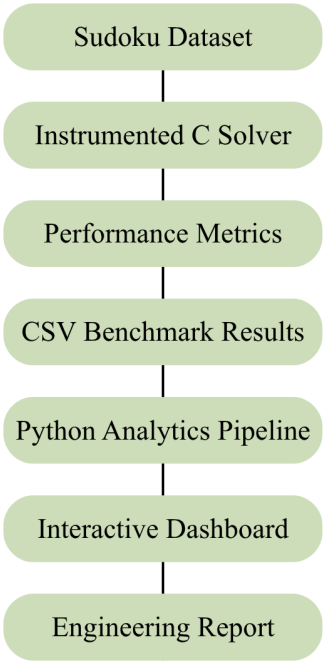


Figure 1. Overall System Workflow

Each stage of the workflow contributes to transforming raw algorithm execution into actionable engineering insights. Rather than terminating after producing a valid Sudoku solution, the system continues through successive stages of data processing, analysis, visualization, and reporting, enabling a comprehensive examination of solver behavior across a large benchmark dataset.

Algorithm Design

The computational core of this framework is an instrumented recursive backtracking solver implemented in the C programming language. C was selected because it provides predictable execution characteristics, low runtime overhead, and direct control over memory, making it well suited for empirical performance measurement. By minimizing abstraction layers introduced by higher-level languages, the implementation allows recorded performance metrics to closely reflect the behavior of the search algorithm itself.

The solver represents each Sudoku puzzle as a fixed-size 9×9 integer matrix, where populated cells contain their assigned values and empty cells are represented by zero. This representation provides constant-time access to individual cells and supports efficient traversal during recursive search.

The solving process follows the classical recursive backtracking paradigm. At each recursive step, the algorithm locates the next unassigned cell and systematically evaluates candidate values from 1 to 9. Candidate values are accepted only if they satisfy the row, column, and subgrid constraints defined by Sudoku. When no valid candidate exists, the algorithm backtracks to the previous decision point and explores the remaining alternatives. This process continues until a complete valid solution is constructed or all possible search paths have been exhausted.

Unlike conventional Sudoku implementations that terminate after producing a solution, the solver developed in this project is instrumented to record detailed execution statistics throughout the search process. Every recursive invocation, candidate evaluation, assignment, and backtracking operation contributes to a comprehensive performance profile for the puzzle under consideration. Consequently, the solver functions not only as a search algorithm but also as an experimental instrument for measuring computational behavior.

The overall architecture separates solving, measurement, and analysis into independent stages. The C solver is responsible solely for algorithm execution and metric collection, while subsequent processing, statistical analysis, and visualization are delegated to the Python analytics pipeline. This separation of concerns simplifies experimentation, improves reproducibility, and allows the analytical components to evolve independently of the solver implementation.

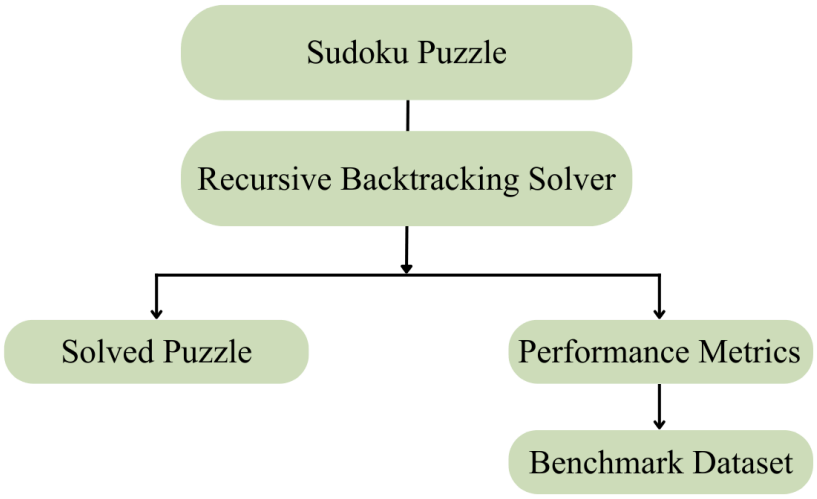


Figure 2. Instrumented architecture illustrating the separation of solving, metric collection, and benchmark data generation.

Dataset Generation

The experimental evaluation performed in this study required a sufficiently large and diverse collection of Sudoku puzzles to ensure that observed performance characteristics were representative rather than anecdotal. Instead of relying on a limited collection of manually selected puzzles, the benchmark dataset was generated programmatically to provide consistency, scalability, and reproducibility.

Puzzle generation was performed using the **Dokusan** Python library, an open-source Sudoku generator capable of producing valid puzzles with varying levels of difficulty. Automated generation ensured that all benchmark instances adhered to standard Sudoku constraints while eliminating inconsistencies that may arise when combining puzzles from multiple external sources.

A benchmark dataset containing **1,000 Sudoku puzzles** was constructed and divided into four difficulty categories: Easy, Medium, Hard, and Expert. This balanced distribution enabled comparisons of solver behavior across progressively more challenging search spaces while maintaining sufficient sample sizes for statistical analysis.

Each generated puzzle was exported in a structured comma-separated values (CSV) format before being processed by the instrumented C solver. During benchmarking, every puzzle was solved independently, producing a corresponding record of execution metrics. The resulting dataset therefore contains both the original puzzle representation and the associated computational characteristics observed during solving.

The use of a standardized benchmark dataset provides several advantages. First, it enables repeatable experimentation by ensuring that identical puzzle sets can be evaluated across multiple solver implementations. Second, it supports quantitative analysis by generating sufficiently large sample sizes for descriptive statistics and regression modeling. Finally, it establishes a consistent experimental foundation upon which future algorithmic comparisons can be conducted.

The benchmark generation process implemented in Sudoku Intelligence Lab is illustrated in Figure 3.

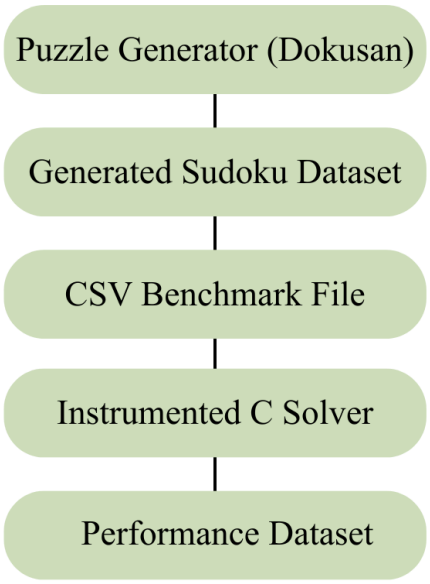


Figure 3. Benchmark Dataset Generation Pipeline

Performance Instrumentation

Performance evaluation requires more than measuring execution time alone. Runtime provides a single summary of algorithm performance but offers limited insight into the computational processes responsible for that performance. To enable a detailed examination of solver behavior, the recursive backtracking implementation was instrumented to record multiple execution metrics throughout the search process.

Each recorded metric captures a distinct aspect of recursive search, allowing computational effort to be analyzed from multiple perspectives rather than through a single aggregate measure. Collectively, these measurements provide a comprehensive representation of algorithm behavior for every benchmark puzzle.

The recorded metrics are summarised in Table 1.

Metric	Engineering Interpretation
Recursive Calls	Total number of recursive function invocations, representing the overall size of the explored search tree.
Backtracks	Number of search states abandoned due to constraint violations or exhausted candidate values.
Candidate Checks	Total number of candidate values evaluated against Sudoku constraints during search.
Successful Assignments	Number of candidate values successfully placed into the puzzle grid.
Failed Assignments	Number of rejected candidate values that violated one or more Sudoku constraints.
Maximum Recursion Depth	Deepest level of recursion reached during execution, indicating the longest search path explored.
Execution Time	Total elapsed time required to solve an individual puzzle.
Empty Cells	Number of initially unassigned cells, used as a structural indicator of puzzle.

Table 1: Performance Metrics Recorded During Solver Execution

Unlike conventional benchmarking approaches that emphasize runtime alone, the proposed instrumentation enables the computational cost of recursive backtracking to be decomposed into individual components. This provides a richer understanding of solver behavior and permits statistical relationships between algorithmic operations and execution time to be investigated.

All recorded measurements are exported alongside the corresponding puzzle identifiers into a structured benchmark dataset. Each record therefore represents a complete experimental observation consisting of both puzzle characteristics and solver performance metrics. This dataset forms the foundation for all subsequent statistical analysis, regression modeling, prediction experiments, and visualization presented in this report.

Experimental Methodology

The experimental evaluation was performed using a benchmark dataset of 1,000 Sudoku puzzles processed under identical execution conditions. Each puzzle was solved independently by the instrumented C solver, which automatically recorded the execution metrics described in the previous section.

The generated benchmark dataset was analyzed using a Python-based workflow consisting of four stages:

1. Data preparation and validation.
2. Descriptive statistical analysis.
3. Regression and prediction modeling.
4. Visualization through an interactive Streamlit dashboard.

All experiments were executed using the same solver implementation, instrumentation strategy, and measurement process, ensuring that observed differences in performance were attributable to puzzle characteristics rather than implementation changes.

Experimental Environment

Parameter	Value
Programming Language	C (solver), Python (analytics)
Dashboard Framework	Streamlit
Visualization Library	Plotly, Matplotlib
Dataset Size	1,000 puzzles
Difficulty Levels	Easy, Medium, Hard, Expert
Operating System	Windows 11
Compiler	GCC
Benchmark Output	CSV

Statistical Analysis

The benchmark dataset was analyzed using descriptive statistical techniques to investigate the computational characteristics of the recursive backtracking solver. Summary statistics, frequency distributions, box plots, and correlation analysis were used to examine the relationships between recorded performance metrics across the benchmark dataset.

The analysis focused on identifying trends in recursive calls, execution time, backtracking behavior, and puzzle difficulty. Comparative analysis across the four difficulty categories enabled the study of how computational effort changed with increasing puzzle complexity. The resulting visualizations and statistical summaries provided the foundation for the regression and prediction studies presented in the following chapters.

Regression Analysis

To investigate relationships between recorded performance metrics, linear regression models were developed using the benchmark dataset. Two regression studies were conducted. The first evaluated whether the number of empty cells could predict recursive search effort, while the second examined the relationship between recursive calls and execution time.

The experimental results demonstrated that puzzle density alone is a weak predictor of computational complexity. In contrast, recursive call count exhibited a strong linear relationship with execution time, producing a high coefficient of determination ($R^2 \approx 0.97$). These findings indicate that internal algorithmic behavior provides a significantly better estimate of computational cost than structural puzzle characteristics alone.

Figure 4 presents the regression models and their corresponding goodness-of-fit measures.

Figure 4. Comparison of the two regression studies conducted in this work. The first model demonstrates that puzzle density is a poor predictor of recursive search effort, whereas the second shows a strong linear relationship between recursive calls and execution time.

Performance Prediction

Following the regression analysis, prediction models were evaluated to estimate solver performance from recorded benchmark data. Newton Divided Difference interpolation was implemented to estimate recursive search effort from puzzle characteristics and validated using Leave-One-Out Cross Validation (LOOCV).

The prediction experiments showed that the number of empty cells alone was insufficient to accurately estimate recursive search effort, yielding poor prediction accuracy ($R^2 = -2.65$). This result suggests that puzzle structure influences solver behavior more strongly than puzzle density. Conversely, regression analysis demonstrated that recursive call count serves as a reliable predictor of execution time, confirming that algorithmic behavior is a more meaningful indicator of computational cost than simple puzzle statistics.

The prediction study reinforces the importance of empirical measurement over heuristic assumptions when evaluating recursive search algorithms.

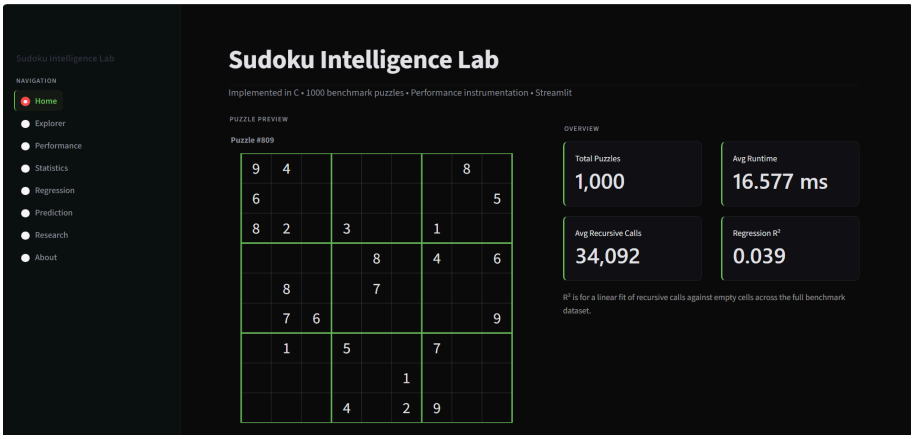
Dashboard Design

To facilitate interactive exploration of benchmark data, an analytical dashboard was developed using Streamlit. Rather than presenting static charts alone, the dashboard enables users to investigate solver behavior through multiple analytical views, including puzzle inspection, performance analysis, statistical summaries, regression studies, and prediction models.

The dashboard is organized into dedicated modules, each focusing on a specific aspect of the benchmark. These modules collectively guide the user from understanding an individual Sudoku puzzle to examining overall computational trends observed across the complete benchmark dataset.

The dashboard integrates data generated by the Python analytics pipeline and presents the results using interactive visualizations developed with Plotly and Matplotlib. This design enables experimental findings to be explored dynamically while maintaining consistency with the engineering workflow described throughout this report.

Figure 5 presents the primary interface of the analytical dashboard.



Experimental Results

The benchmark framework successfully processed a dataset of **1,000 Sudoku puzzles**, generating detailed execution metrics for every benchmark instance. The recorded measurements provided a comprehensive view of solver behavior beyond conventional runtime evaluation.

The statistical analysis revealed that computational effort increased substantially with puzzle difficulty. Harder puzzles consistently required greater numbers of recursive calls, candidate evaluations, backtracking operations, and execution time, indicating a larger search space explored by the recursive backtracking algorithm.

Regression analysis demonstrated that the number of empty cells alone is a poor predictor of recursive search effort. In contrast, recursive call count exhibited a strong linear relationship with execution time ($\mathbf{R^2 \approx 0.97}$), suggesting that internal algorithmic behavior provides a significantly more reliable estimate of computational cost than structural puzzle characteristics.

Prediction experiments based on Newton Divided Difference interpolation achieved limited accuracy ($\mathbf{R^2 = -2.65}$), indicating that puzzle density alone is insufficient for estimating solver complexity. These findings reinforce the importance of empirical performance measurement over simple heuristic indicators.

A summary of the principal findings is presented in Table 2.

Finding	Observation
Benchmark Dataset	1,000 Sudoku puzzles
Solver Success Rate	100%
Strongest Runtime Predictor	Recursive Calls
Regression Performance	$R^2 \approx 0.97$
Prediction Performance	$R^2 = -2.65$
Primary Insight	Puzzle structure influences computational effort more than empty cell count

Engineering Discussion

The objective of this project was not merely to implement a Sudoku solver, but to construct a reproducible engineering framework for analyzing algorithm performance. By instrumenting the recursive backtracking algorithm and integrating automated benchmarking with statistical analysis and visualization, the project demonstrates how algorithm execution can be transformed into measurable experimental data.

One of the most significant findings is that computational effort cannot be accurately estimated from puzzle density alone. Although the number of empty cells provides a simple measure of puzzle structure, the experimental results indicate that it is a poor predictor of recursive search effort. Instead, internal execution metrics, particularly recursive call count, offer a far more reliable representation of computational complexity.

The project also illustrates the value of combining multiple engineering disciplines within a single workflow. Algorithm implementation in C, automated data processing in Python, statistical modeling, and interactive visualization were integrated into a cohesive benchmarking framework. This modular architecture improves reproducibility while allowing individual components to be extended independently.

Although Sudoku served as the benchmark domain, the methodology developed in this work is broadly applicable to other search algorithms and constraint satisfaction problems. The framework can therefore be viewed as a reusable foundation for empirical algorithm analysis rather than a Sudoku-specific implementation.

Limitations

Like any empirical study, this work has several limitations. The benchmark framework evaluates a single recursive backtracking algorithm and therefore does not compare its performance against alternative solving strategies such as Constraint Propagation, Dancing Links (Algorithm X), or heuristic-based search methods. Consequently, the reported findings describe the behavior of the implemented solver rather than Sudoku solving algorithms in general.

The benchmark dataset, although sufficiently large for the scope of this study, consists of programmatically generated puzzles and may not capture the full diversity of manually designed Sudoku instances. In addition, the prediction study relies primarily on puzzle density and recorded execution metrics, limiting its ability to model deeper structural characteristics that influence search complexity.

Despite these limitations, the proposed framework provides a reproducible foundation for empirical algorithm analysis and can be extended to support additional algorithms, datasets, and analytical techniques.

Future Work

Several opportunities exist to extend the capabilities of Sudoku Intelligence Lab. Future work may include benchmarking additional solving algorithms such as Constraint Propagation, Dancing Links (Algorithm X), and heuristic-based search methods to enable comparative performance analysis under identical experimental conditions.

The prediction framework can also be improved by incorporating richer puzzle features and machine learning techniques to better estimate computational effort. Additional benchmark datasets, larger sample sizes, and more diverse puzzle sources would further strengthen the reliability of the experimental findings.

Finally, the modular architecture of the project makes it adaptable to other constraint satisfaction problems, allowing the benchmarking methodology developed in this work to be applied beyond Sudoku.

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. MIT Press, 2022.
[2] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Pearson, 2021.
[3] D. E. Knuth, "Dancing Links," in Millennial Perspectives in Computer Science, Palgrave Macmillan, 2000, pp. 187–214.
[4] B. Weber, "A Tutorial on Constraint Satisfaction Problems," AI Magazine, vol. 31, no. 4, pp. 85–96, 2010.
[5] Dokusan Contributors, "Dokusan: Sudoku Puzzle Generator." GitHub Repository. Available: https://github.com/KeithGalli/dokusan
[6] Streamlit Inc., "Streamlit Documentation." Available: https://docs.streamlit.io
[7] Plotly Technologies Inc., "Plotly Python Documentation." Available: https://plotly.com/python/
[8] Python Software Foundation, "Python Documentation." Available: https://docs.python.org/3/
[9] ISO/IEC 9899:2018, Programming Languages — C.

Appendix

Appendix A - Repository Structure

Sudoku-Intelligence-Lab/

```
├── c_engine/
├── dashboard/
├── python/
├── data/
│   ├── dataset/
│   ├── input/
│   └── output/
├── images/
├── reports/
├── docs/
├── README.md
└── requirements.txt
```

Appendix B - Performance Metrics

The recursive backtracking solver was instrumented to record multiple execution metrics during the solution process. Each metric captures a different aspect of the search algorithm's behavior, enabling a comprehensive analysis of computational effort beyond execution time alone.

Metric	Definition	Significance
Recursive Calls	Total number of recursive function invocations performed while solving a puzzle.	Represents the size of the explored search tree and serves as the primary indicator of search effort.
Backtracks	Number of times the solver abandoned a search path after exhausting all valid candidate values.	Indicates the extent of unsuccessful exploration and reflects the complexity of the search space.
Candidate Checks	Total number of candidate values evaluated against Sudoku constraints.	Measures the amount of constraint validation performed during execution.

Successful Assignments	Number of candidate values successfully placed into the Sudoku grid.	Represents successful progress toward the final solution.
Failed Assignments	Number of candidate values rejected because they violated Sudoku constraints.	Reflects the frequency of invalid search decisions encountered during recursion.
Maximum Recursion Depth	Deepest recursion level reached while solving a puzzle.	Indicates the longest search path explored before reaching a solution or backtracking.
Execution Time (ms)	Total elapsed time required to solve a puzzle, measured in milliseconds.	Provides an overall measure of computational cost and is used for runtime comparison.
Empty Cells	Number of unassigned cells in the initial puzzle configuration.	Serves as a structural characteristic of puzzle density and was evaluated as a potential predictor of computational effort.

Interpretation

No single metric is sufficient to fully characterize the behavior of a recursive backtracking algorithm. While execution time provides an overall measure of performance, the remaining metrics reveal how that time was spent during the search process. For example, two puzzles with similar execution times may exhibit significantly different recursive depths, backtracking frequencies, or candidate evaluations.

Collectively, these metrics enable a detailed empirical analysis of algorithm behavior, supporting the statistical analysis, regression studies, and prediction experiments presented throughout this report.

Appendix C - Dashboard Modules

Module	Purpose
Home	Project overview
Explorer	Puzzle inspection
Performance	Performance distributions

Statistics	Statistical summaries
Regression	Regression studies
Prediction	Prediction analysis
Research	Engineering findings
About	System information